

# Lecture: Graphs

Week 6. From here on, the input is no longer a sequence. It is a set of vertices and a set of edges.

Write  $G = (V, E)$ . An undirected edge is a two-element set. A directed edge is an ordered pair. A weight is a number on an edge, usually a cost.

## Representations

Adjacency list: for each vertex, the neighbours (and weights). Space  $\Theta(V + E)$ . This is the default.

Adjacency matrix: a  $|V| \times |V|$  table. Fine when the graph is dense, or when you need  $O(1)$  “is this edge present?”.

If you store the matrix “because it is simpler” on a road network, you will run out of memory and also look like you skipped this slide.

## Walks you must know

**BFS** from  $s$  discovers vertices in order of unweighted distance. Queue. Tree edges form a shortest-path tree in hops.

**DFS** from  $s$  goes deep, then backtracks. Stack or recursion. Useful for cycle detection, topological sort, and connected components.

**Dijkstra** from  $s$ , non-negative weights: repeatedly settle the unsettled vertex with smallest distance. With a binary heap this is  $O((V + E) \log V)$ . Negative weights: do not use it; that is Bellman–Ford territory.

A common exam trap: BFS is not Dijkstra with all weights equal \*if you implement BFS on a weighted graph\*. Equal weights plus Dijkstra is fine. BFS ignores the numbers.

## Project checkpoint (read this)

The graph project is a shortest-path viewer over a campus map. You do not submit the full project this week.

The graph project checkpoint is due 28 November 2026. Upload a working BFS and Dijkstra on the sample graph, plus a screenshot of one query. No write-up yet.

If your Dijkstra disagrees with BFS on a unit-weight graph, fix that before you add fancy drawing. The drawing is the easy part.